



Simulating mixed agile and plan-based requirements prioritization strategies: proof-of-concept and practical implications

Daniel Port & Tung Bui

To cite this article: Daniel Port & Tung Bui (2009) Simulating mixed agile and plan-based requirements prioritization strategies: proof-of-concept and practical implications, European Journal of Information Systems, 18:4, 317-331, DOI: [10.1057/ejis.2009.19](https://doi.org/10.1057/ejis.2009.19)

To link to this article: <https://doi.org/10.1057/ejis.2009.19>



Published online: 19 Dec 2017.



Submit your article to this journal [↗](#)



Article views: 557



View related articles [↗](#)



Citing articles: 3 View citing articles [↗](#)



Simulating mixed agile and plan-based requirements prioritization strategies: proof-of-concept and practical implications

Daniel Port,
Tung Bui

*Department of Information Technology
Management, Shidler College of Business,
University of Hawaii, Manoa, U.S.A.*

Correspondence: Daniel Port,
Department of Information Technology
Management, Shidler College of Business,
University of Hawaii, Manoa, U.S.A.
Tel: +1 808 956 7430;
Fax: +1 808 956 9889;
E-mail: dport@hawaii.edu

Abstract

In this paper, we address the efficacy and pragmatics of mixing two primary strategies for requirements prioritization in order to incorporate the benefits of both plan-based (PB) and agile development methods while avoiding their drawbacks. As it is intractable to directly study the performance of strategies on real projects, we conducted a comprehensive empirically based simulation under a variety of conditions of requirements dynamism, project size, and duration. Simulation results suggest that a mixed strategy for requirements prioritization seems to work best in all but cost for typical levels of dynamism on average. Our findings also indicate that, as theorized, PB and agile strategies perform well within opposite extremes of dynamism. However, they do not outperform the mixed strategies even within their home grounds – that is large and complex systems with stable requirements for PB, and small and dynamic projects for agile methods. Given the unknown, unknowable, or variable nature of dynamism and the dramatic differences in effectiveness for agile and PB strategies under extreme development scenarios, a mixed strategy appears to yield the best results overall. We introduce two mixed strategies – simply adding cost–benefit (CB) to the agile approach, and a more sophisticated ‘hybrid’ (HY) approach that modulates development iteration size to maximize the expected CB for each iteration. We propose a step-by-step method to implement this HY strategy. We provide a structured analysis of the benefits and assumptions of agile and PB requirements prioritization methods (e.g., Pareto optimization), and outline a framework for analyzing and assessing the effectiveness of strategies including several new metrics. This research can furthermore serve as a framework for future validation of the proposed mixed strategies using actual software projects.

European Journal of Information Systems (2009) 18, 317–331.

doi:10.1057/ejis.2009.19; published online 21 July 2009

Keywords: requirements prioritization; agile prioritization; requirements engineering; strategic and development planning; integration of plan-based and flexible/agile processes; risk management

Introduction

It is a widely recognized fact that for most information system (IS) development projects, not all requirements will be implemented, and that some form of requirements prioritization is essential (e.g., Regnell *et al.*, 2001). With high customer expectations, tight delivery schedules, and limited resources, prioritization has become a common strategy to limit the scope (e.g., Siddiqi & Shekaran, 1996), and deliver the most essential functionalities as early as possible (Wiegers, 1999).

Received: 5 May 2008
Revised: 13 October 2008
2nd Revision: 28 May 2009
Accepted: 16 June 2009

There has been considerable interest in requirements prioritization from both the agile and plan-based (PB) perspectives. For example, Lubars *et al.* (1993) provide a comprehensive review of the state of practice in requirements modeling that focuses on prioritization, whereas Beck (2001) notes the challenging role of prioritization in extreme programming and Patton (2008) underscores the importance of prioritizing with respect to explicit goals on agile projects. Aurum & Wohlin (2003) highlight the role of decision making in prioritization within the context of the highly active field of requirement engineering (RE) processes. PB (sometimes called disciplined or plan-driven) development approaches are the principle adopters of RE methods and tools and hence requirements prioritization (Hoffmann and Lehner, 2001). In this paper, PB and RE-based prioritization methods are considered synonymous. We note, however, that agile enthusiasts also embrace requirements prioritization as a fundamental activity. Indeed, the majority of methods described in a comprehensive survey of agile methods (Abrahamsson *et al.*, 2002; Cao & Ramesh, 2008) include some form of requirements prioritization.

Justifiably, the literature also acknowledges that requirements prioritization is a very complex and costly activity (e.g., Lubars *et al.*, 1993; Karlsson & Ryan 1997; Boehm & Turner, 2004). The development of large distributed systems, in particular, often exacerbates cost and complexity greatly and, tends to render this important activity futile (Damian & Zowghi, 2002). Perhaps, in part, a reaction to such issues, a well-documented alternative to the plan-driven development approach, is an agile methodology. As discussed later in this paper, the basic concept of agile methods is germane to that of prototyping. It can be seen as a collaborative approach by self-organizing teams to deal with iterative and incremental software development to meet the evolving and changing needs of the users. Although agile methods are designed to offset most of the drawbacks of their PB counterparts, they have their own set of challenges: the non-trivial effort to coordinate among team members, the potential impasse in reaching a consensus in requirements prioritization, and as will be shown later, risky early implementation commitments that degrade global prioritization benefits. When these challenges are handled properly, one would expect better performance, thanks to increased communications and collaboration.

Outside of the dogmatic debate regarding agile and PB approaches to software development, most academics and practitioners recognize that *both* approaches have their merits and excel in their respective home ground (Boehm & Turner, 2004). Boehm & Turner (2004) use the term 'home ground' to refer to the development conditions and environment that seems most favorable for a development method. PB methods seem to be most appropriate for large and complex systems with stable requirements and predictable environment, whereas agile methods seem to be more appropriate for systems with

volatile requirements and small development teams. As most real-life projects exhibit characteristics that are not purely typical to the either PB or agile home grounds, Boehm & Turner (2004) suggest that a *mixed* strategy is generally more suitable. They argue that when a project has more dynamic characteristics, agile approaches are more fitting; conversely, when the project appears to be more stable or controllable (or mandated to be so such as with safety critical systems), PB approaches are recommended. One major issue is that it is generally difficult to determine in advance where a project lies in this stable/dynamic spectrum. This challenge is particularly prevalent when the degree of requirements volatility is either unknown or unknowable at any given time (as is generally the case). This is one of the major rationales given by agile protagonists for using the agile approach. Yet, PB approaches are still pervasive (Siddiqi & Shekaran, 1996), as they offer the perception of a greater controllability and optimization.

Given this and numerous other related considerations, what is a practical means for investigating the effectiveness of requirements prioritization? Comprehensive simulation appears to be an attractive option for investigating and providing empirical support, and justification of new software engineering methods whose effectiveness measures are intangible and unobservable (Kleindorfer *et al.*, 1993). Such simulations are commonplace and accepted in the management and operations research literature where the evaluation challenges are highly analogous to those in software engineering (Oren, 1981; Balci, 1995; Sargent, 1998).

We propose a simulation model that is a literal representation of agile and PB methods as they are defined in the literature. As presented in this paper, our prioritization strategies are modeled on empirical results from (Karlsson, 1996; Boehm, 2003; Paetsch *et al.*, 2003; Cao & Ramesh, 2008). To Boehm & Turner (2004), the general perception is that development practice follows one of the two extremes – either agile or PB. As we are interested in investigating properties of prioritization strategies, we must consider what is considered to be at the extremes. We suspect that the strategies used in practice are HYs that reflect this but without explicit foundational premise. Thus, we seek to provide here a foundation and refinements for this practice along with clear guidance in its application.

The paper begins with an overview of the benefits and limitations of PB and agile requirements prioritization as a strategic development process activity. We seek 'HY' requirements prioritization strategies that leverage the globally optimizing benefit of PB methods, while maintaining the flexibility and low overhead of the agile approaches. The simulation is used to highlight the benefits of mixed approaches. Our methodological discussion provides the necessary basis to enable the practitioner to make use of whichever of the multitude of methods and processes that are most suitable in their particular context. We conclude with a summary of the

findings and contributions of this paper, highlight key limitations, and suggest directions for future research.

A strategic perspective of PB and agile requirements prioritization

PB or plan-driven strategies operate under three key assumptions. First, the complete set of requirements of the system-to-be is well known and understood before implementation. Second, with a great deal of due diligence in interacting with users, developers should be able to judge the value (defined as the expected benefit to the system if the requirement is implemented) and costs associated with each of these requirements. Third, it is expected that there is little change in the requirements during development or that changes will not occur given enough advanced analysis and planning. Under these circumstances, it should be relatively easy for the project managers to prioritize and plan an optimal implementation effort. To further help measure, the benefits and costs more accurately and more timely, a number of techniques have been devised. Some developments include the use of information and planning software agents that use case-base planning and reactive execution to assist developers articulate their client's goals and develop the plans to meet them (e.g., Cockburn, 2000; Hammond & Burke, 1997; Cao & Ramesh, 2008).

The fundamental basis of a PB strategy is to take advantage of Pareto optimization by implementing the highest cost-benefit (CB) requirements first, allowing lower leverage requirements to be dropped if development ends earlier than planned or if it is not possible to implement all requirements. The prioritization activity begins with the analysis and discussion of the candidate software requirements. After this, the stakeholders collaboratively determine for each individual candidate requirement what the cost of implementing the requirement would be and how much value the requirement contributes to the overall system (Karlsson & Ryan, 1997). The assessment of values and costs for the requirements can be performed using any ranking or comparison algorithm, such as AHP (Saaty, 1990) that uses pairwise assessment of 'candidate requirements. AHP is a popular prioritization technique, thanks to the ease of doing pairwise comparisons. For example, a value of 3 for $\text{Cost}(R_1 \text{ and } R_2)$ indicates that requirement 1 is valued three times as high as requirement 2. Reciprocally, $\text{Cost}(R_2 \text{ and } R_1)$ would then have value however, there are other common techniques, such as ordinal/cardinal rankings, sums-of-the-ranks, and outranking relations (e.g., Sivzattian & Nuseibeh, 2001) the choice of the prioritization techniques does affect the outcomes (Bui, 1987). However, it does not affect the generalization of the mixed-based approach. The cost-value approach in Karlsson & Ryan, (1997) propose a five-step cost-value approach to determine priorities among candidate requirements, and how this prioritization can be used in the development process. This approach seems to be effective in projects known to have little or no

requirements volatility (also called dynamism), and by the time the development ends, the goal is that the majority of high-leverage requirements get implemented, i.e., the '80%' of the value that resided in '20%' of the requirements (quotes added to emphasize that these are only representative percentages). This is often referred to as the 'Pareto principle' or '80-20' rule. It is shown in Port *et al.* (2007) that the Pareto ordering that optimizes overall cost and value through this principle is determined by the sequencing activities (e.g., requirements implementation) by the highest to the lowest CB (defined as value/cost).

Given the best development circumstances – well-defined requirements and no expected change (hereafter, we call this highly deterministic scenario 'no-dynamism'), it is expected that the Pareto-based PB strategy would lead to an overall optimal implementation plan. Unfortunately, the reality is far from deterministic; requirement changes are inevitable, and requirements evolve more than most project managers can control.

To deal with this reality, the proponents of agile methodologies argue that there are more effective ways of developing software through the use of processes and tools that help self-organizing teams to work together and respond swiftly to changed requirements (e.g., Siddiqi & Shekaran, 1996). Highest values through customer's satisfaction can be achieved, thanks to sustained and frequent deliveries of working software. Furthermore, to support changes, intense collaboration is the norm within small successive iterations, with frequent reviews (e.g., Poppendieck & Poppendieck, 2003). Ceschi *et al.* (2005) conducted a survey among 20 managers of PB and agile companies, and contend that agile methods *seem* to offer a good solution for improving the project management process as well as the quality of customers' interaction and satisfaction.

However, despite growing interest, there appears to be a lack of published papers that discusses requirements prioritization within agile development. Perhaps, this is not surprising given that agile methods promote flexible implementation planning artifacts, such as stories helping requirements to become more concrete or less ambiguous (Beck, 2001). However, this practice tends to incur additional development costs. However, recently, there has been greater emphasis on more global prioritization in planning agile development, such as Cohen (2005) and Patton (2008).

Agile development methodologies view collaborative prioritization fundamentally as a *tactical* activity (Beck, 2001) to be performed within each development iteration or timebox. Requirement changes are integral to the development process. Agile prioritization approaches, such as the planning game (Beck, 2001), simplify and lower the prioritization effort, and are apparently workable within a distributed development context (Ramesh *et al.*, 2006); they fail to leverage the 80/20 rule benefits of PB approaches as discussed previously. Specifically, an agile approach generally does not attempt to discover a

complete set of requirements from the outset in preference of letting them 'emerge' during the development. Priorities for small sets of requirements are set locally within each development iteration, whereas a PB method will prioritize globally within the overall project or perhaps a portfolio of projects. It is unrealistic and somewhat contrary to the premise that change is inevitable to assume that all the most important requirements will emerge early within the development. So, although the agile approach is highly responsive to change, the risk is that a high leverage requirement (i.e., high CB) may go unrecognized or recognized too late, and not get implemented in favor of lower leverage requirements. Such phenomenon has been reported in numerous accounts of agile development efforts (Boehm, 1991; Boehm & Turner, 2004).

Mixed strategies

Intuitively, there should be a mixed approach to build on the unique strengths, and reduce the weaknesses of both approaches (Boehm & Turner, 2004). There are a number of interesting ways to mix PB and AG. In theory, we envision two possible strategies in the search of an effective mixed strategy and compare these with PB and AG (Table 1).

The main tradeoff consideration is effort. Adding PB activities to an agile approach or vice versa will invariably increase the effort needed to perform prioritization. Hence, this is another consideration when comparing new strategies. If a mixed strategy has better performance, it should be beneficial enough to outweigh the additional effort required use it. We consider mixture approaches from two opposite perspectives – adding PB activities to AG and adding agile activities to PB:

Adding PB activities to AG – the AG2 strategy: The PB strategy prioritizes by the highest CB first to reduce the risk of not implementing a high-leverage requirement. In a highly dynamic environment where estimated benefits or values may change, what may once have been a high-leverage requirement (i.e., a high value for low cost) may change to a low-leverage requirement (or vice versa),

thereby, obviating the benefit of this prioritization. Hence, a natural variation of AG to explore is to order the requirements within each iteration by CB (defined as value/cost) rather than by value alone. The effort increase for AG2 is that a cost assessment must be performed in addition to the usual value assessment.

Adding agile tactics to PB – the HY strategy: The AG strategy is known to be effective when there is high dynamism because it enables adjustment of priorities when requirement values change. The shortcoming of this strategy is that it assumes that implementing the highest valued requirements that are known within each iteration will result in implementing the highest valued requirements over all the iterations. That is, local value prioritization could lead to global value prioritization. Generally, as discussed previously, this situation is not likely, and the result will be a suboptimal prioritization with respect to cost-effectiveness of the overall development. This is also true regardless of what local prioritization method is used, for example, CB rather than value as in AG2. An alternative might be desired. The argument here is that the iteration sizes in an agile approach do not account for the likelihood of higher leverage requirements appearing in a later iteration. Agile methods tend to fix the (usually small) size of the iteration, through timeboxing or otherwise, and then implement as much as possible in that iteration then carrying the remainder to the next iteration. Because of this, the iteration may include implementation of requirements after a point of 'diminishing returns' or where the marginal cost exceeds the marginal value (in that iteration). The reaction to this may be to shorten the iterations as illustrated in Figure 1 where each tick mark represents the cumulative cost and cumulative value up to implementation of that requirement (this is called the *strategy profile*, which we will elaborate on later). But, how short should the iterations be? Shorter iterations increase effort due to review and replanning overhead, and can rapidly become impractical. Longer increments may fail to respond to emergent or changed requirements.

Table 1 Four possible requirements prioritization strategies

Strategy	Key approach
Plan-based (PB)	Traditional or plan-driven approach with known requirements and comprehensive cost-benefit analysis and established development plan
Agile (AG)	Periodic teamwork with focus on 'value-up' through quick delivery of working software, unknown requirements, and frequent reprioritization
Agile plus (AG2)	Agile approach with consideration of cost in addition to value
Hybrid (HY)	A combination of PB and AG2 with variable iteration sizes

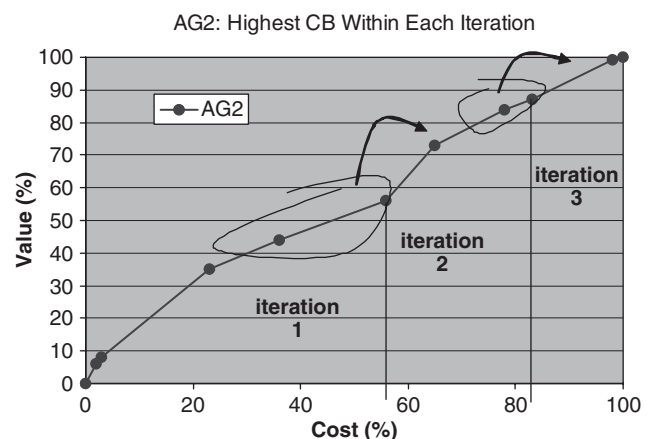


Figure 1 Cutting iterations short in AG2.

At least in principle, the key advantage of the PB strategy to avoid this issue is to take a global assessment of requirements from the start and optimize them by prioritizing high leverage requirements first, and then attempt to minimize dynamism as the development progresses. In general, such a strategy is not necessarily achievable, and it is risky in a high-dynamism environment.

As an alternative, we propose a HY strategy that performs an initial comprehensive requirements assessment to establish an overall initial prioritization, and then, on subsequent iterations, this prioritization is updated according to CB reassessments at intervals, which minimize CB leverage risks (i.e., implementing a lower leverage requirement in favor of a high-leverage one). The key is to choose a *strategic stopping point* within the iteration that implements the majority of the high-leverage requirements and then leaves the rest for the next iteration where they can be reprioritized according to a new cost and value assessment. A natural strategic stopping point within a given iteration is when the total marginal cost exceeds the total marginal value for the requirements in a given iteration (i.e., 'economic turning point'). This point can be shown to be the precise location that minimizes CB leverage risk under the dynamism conditions that we have described (e.g., normally distributed value changes and Poisson arrival of new requirements).

This HY approach is a little more involved than the strategies discussed previously, so we now list the process steps involved to help clarify:

- (1) *Establish a small number of key stakeholders to form an 'agile priority assessment taskforce.'* The members of this taskforce must include at least one customer representative, and one developer manager. It is essential that the taskforce be limited in size as much as possible. In the event that there are many distributed development sites, consider combining representation from sites that serve largely in the same capacities as other sites together and excusing sites with minor roles in the project.
- (2) *Generate a candidate list of 'base requirements' from the current collection of stories.* These are not formally specified requirements, rather they are brief descriptions of expected system features. The list should be as comprehensive as possible and include speculative requirements that may be added in the future. Call these candidate requirements $R = \{r_1, r_2, \dots, r_n\}$.
- (3) *Use a ranking technique to collaboratively assess costs, $Cost(r_i)$ and values, $Value(r_i)$ for all the candidate base requirements r_i in R .*
- (4) *Generate an 'implementation strategy template' based on the prioritization results that consists of a list of candidate feature requirements, their estimated costs and values ordered by the highest to the lowest $CB = Value(r)/Cost(r)$.*
- (5) *Review the implementation strategy template at the beginning of the development iteration.* Removal of

non-relevant feature requirements to their particular development effort is made. Cost and value assessments are adjusted and new requirements are added if necessary. Call the revised requirement set $S = \{r_1, r_2, \dots, r_d\}$ and order them according to the highest CB.

- (6) *Compute $X(S) = \sum_{r \in S} Cost(r)$ and $Y(S) = \sum_{r \in S} Value(r)$, the total cost and value of the revised requirements set, respectively.* Set the iteration size to implement all the requirements where $CB > Y(S)/X(S)$ and an agile implementation plan is established for the current iteration. If $Y(S)/X(S)$ is nearly the same as the previous iteration (say within 10%), then this is the final iteration. Note that if 'normalized' costs and values are used, scales of the total cost and total value can be normalized to 100% for the current set of requirements, then $Y(S)/X(S) = 1$ for each iteration.
- (7) *After the iteration completes, remove implemented requirements from S and go again to step 5.* This repeats until development effort ends or there are no requirements are left to implement.

A specific example of performing the HY strategy and its comparison to other strategies will be given in the next section.

Analysis of requirements prioritization strategies

From the somewhat abstract discussions above, it is not obvious how effective a strategy or how strategies can be compared. To address this kind of analysis, we will make use of the *strategy profile*. A strategy profile is a graph of the partial sums (i.e., cumulative) of the independent variables used to determine a strategies' ordering. Variables that use different strategic goals typically include risk exposure, risk reduction, schedule, effort, quality, and so forth. In our cost-value example, the variables are simply cost and value. A strategy implies an ordering of requirements $\{r_1, r_2, \dots, r_n\}$ where r_i has higher priority over r_j when $i < j$ and the partial sums are defined for this order as

$$C_k = \sum_{i=1}^k Cost(r_i), \quad V_k = \sum_{i=1}^k Value(r_i).$$

The strategy profile is now created by plotting the points $(C_1, V_1), (C_2, V_2), \dots, (C_n, V_n)$. Each i th tick mark on this graph corresponds to the i th requirement r_i in the strategy. Note that strategy profiles, similar to probability distributions, will always be *non-decreasing*. Figure 1 is an example strategy profile for AG2.

A working example of requirements prioritization strategy analysis

To illustrate the analysis of strategies, we will make use of a simple example of the popular cost-value approach as adapted from an example reported in Wikipedia (http://en.wikipedia.org/wiki/Prioritizing_Requirements_using_a_Cost-Value_ApproachOther_Prioritization_Techniques, accessed on September 25, 2008). Suppose that the values and costs in Table 2 have been identified for the

Table 2 Example percentages for value–cost of candidate requirements

Requirement R_i	R_1	R_2	R_3	R_4	R_5	R_6	R_7	R_8	R_9	R_{10}
Value V_i	12	6	2	3	27	9	1	17	11	12
Cost C_i	20	2	1	5	20	13	2	9	13	15

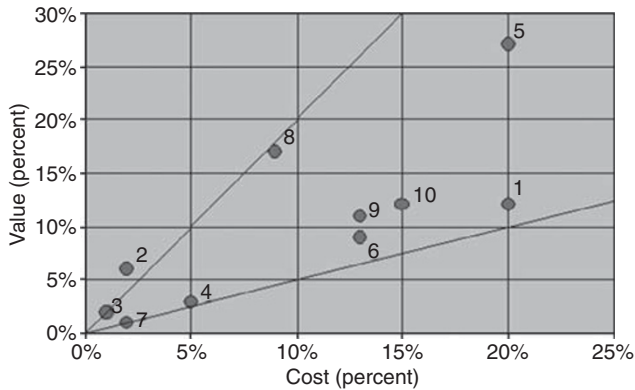


Figure 2 Scatter diagram of cost–value assessments or requirements. The lines represent constant cost–value combinations of 2 and 0.5, respectively.

candidate requirements as per the cost–value approach, such that table might just also as easily be the result of Step 2 in the HY process discussed above or as a similar process step for other strategies.

The goal of the cost–value approach is to derive an implementation strategy that lines up effort to address the highest value/lowest cost (or best cost–value) requirements first with the expectation that implementation risk is reduced. As illustrated in Figure 2, a scatter diagram of the costs and values suggests that requirements R_2 , R_3 , and R_8 have the highest implementation priorities (upper left area of the figure).

The conclusion is that one might place the requirements into three ‘priority bins’ according to where they are located with respect to the lines of constant cost–value. The bins are implemented in three increments, starting with requirements R_2 , R_3 , and R_8 for iteration 1, then R_1 , R_5 , R_6 , R_9 , and R_{10} for iteration 2, then the remainder in iteration 3. Such priority bins are similar to the high–medium–low prioritization categories for the Planning Game used with Extreme Programming (Beck, 2001). But, we note that for this, cost is generally not explicitly considered. The cost–value prioritization binning approach is popular in practice. From a strategic perspective, this arrangement opens up, however, a number of important practical questions. They include:

- (1) Was it a good choice to use the constant cost–value combinations 2 and 0.5 to partition the high and low priority requirements?

- (2) What confidence do we have that the strategy will achieve our risk reduction goal?
- (3) How do the quality and accuracy of our value and cost assessments effect achieving our risk reduction goal?
- (4) What is a necessary amount of requirements to implement and are these covered in the high priority requirements?
- (5) How can we ensure that it is feasible to implement the high priority requirements?
- (6) If we do not know in advance where we will have to stop implementing, how can we ensure that we could not have done better with another strategy?
- (7) Would having a fixed budget change our strategy?
- (8) How is the strategy updated throughout the development lifecycle in the face of inevitable changes (e.g., requirements volatility)?

To illustrate the significance of these questions, we will consider slightly different strategies for ordering the requirements within each bin. By plotting strategy profiles, we see that different strategies have significantly different outcomes. To help better understand the impact of different strategies, we will compare each with a baseline ‘arbitrary’ strategy.

The arbitrary strategy, or perhaps more accurately, ‘no strategy’ is simply a random ordering of the requirements. This strategy profile typically looks approximately linear. Given that at each point, the value and the cost are randomly distributed, so the average C_k equals the average V_k (in the normalized cost and value case as in our current example), giving an expected rate of increase of 1 and hence is roughly linear. As the order of the requirements listed in Table 2 was selected arbitrarily, this ordering is as good as any other random order and it was used to generate the arbitrary strategy in Figure 3.

The arbitrary strategy (i.e., having no strategy) serves as a useful neutral baseline. It is neither a good nor a bad strategy, and requires no effort to perform. As such, any strategy that incurs effort should perform better than the baseline. In the following examples, we assume the requirements do not change in value or cost, and no new requirements are added during the development. Figure 3a illustrates the strategy profile of the basic three cost–value priority bins, which we call ‘aggregated’ because it assumes the requirements in each bin are implemented together without any order. It has little difference from the arbitrary strategy and thus has questionable benefit for the effort in performing it. With similar results, Figure 3b illustrates an example of arbitrary ordering of the requirements within each bin. By ordering the requirements in each bin from the highest to the lowest value, as indicated in Figure 3c, we see some improvement over the arbitrary strategy, but depending on where the development ends, it may perform worse. Figure 3d shows a consistent improvement over the arbitrary strategy when ordering in each bin is done from the highest to the lowest CB (remember,

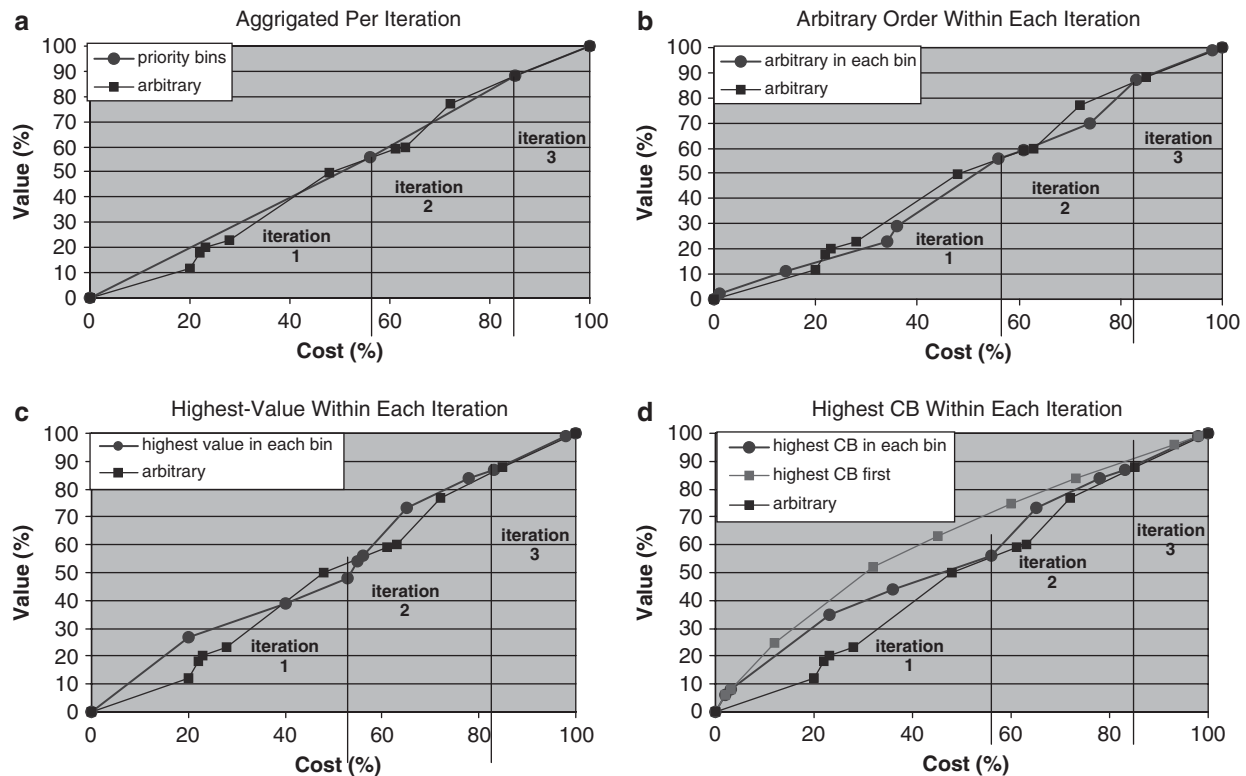


Figure 3 A comparison of strategy profiles of variations for cost-value approach.

this is value/cost). As a prelude to the next discussion, we see that there are better strategies than this as illustrated by the top curve in Figure 3d. This strategy is generated by forgetting the bins and ordering by the highest to the lowest CB, giving the so-called 'Optimal' Pareto strategy.

The example in Figure 3 suggests that the simple cost-value approach may not be effective without further consideration of ordering within each priority bin. Using the data in Table 2, we compare a number of other relatively simple and PB strategies. Figure 4a considers the strategy of 'implementing the lowest cost requirements first.' Constrained by limited resources, many organizations are tempted to adopt this strategy without any regard to their relative values. But, this may not be a wise strategy given that it cannot be guaranteed to perform better than an arbitrary strategy. Indeed, in our simple example, we see that after about 65%, the arbitrary strategy actually outperforms this strategy. In general, prioritizing by the lowest cost is risky with respect to expected value creation. A focus on value appears at first to improve this picture, but ultimately, it is not much better as seen in Figure 4b. Both cost and value need to be factored in to consistently outperform the arbitrary strategy. There are two simple candidates to examine, order by the benefit (B defined as value-cost) and order by CB (as before, defined as value/cost). Figure 4c and d illustrate a consistent and reliable improvement over the arbitrary strategy. It can be shown that, as mentioned

previously, ordering by CB will result in the Optimal Pareto strategy, which no strategy can outperform (under these simple conditions).

Although not shown in Figure 4, there exist strategies that are truly 'bad' with respect to the arbitrary strategy. Prioritizing the requirements according to the lowest CB is such an example and is obviously not worth considering. A bad strategy may result in inefficient use of implementation resources with respect to creating value. Typically, requirement implementation efforts are terminated when an arbitrarily defined budget is expanded (rather than all requirements satisfied and tested), or when stakeholders 'feel' as though enough of the system has been implemented, or more commonly, when higher priority is given to project areas, for example, transition, installation, and marketing. The result is a high degree of risk due to uncertainty in what the value of the system as implemented provides.¹

An example of the HY method

The example strategy profiles in Figures 3, and 4 are essentially 'PB' and perhaps overly simplistic in many regards. However, they illustrate well the potential differences that the choice of strategy can make. One unrealistic assumption in these examples is that there is

¹We have been documenting several cases in industry where this phenomenon has actually occurred.

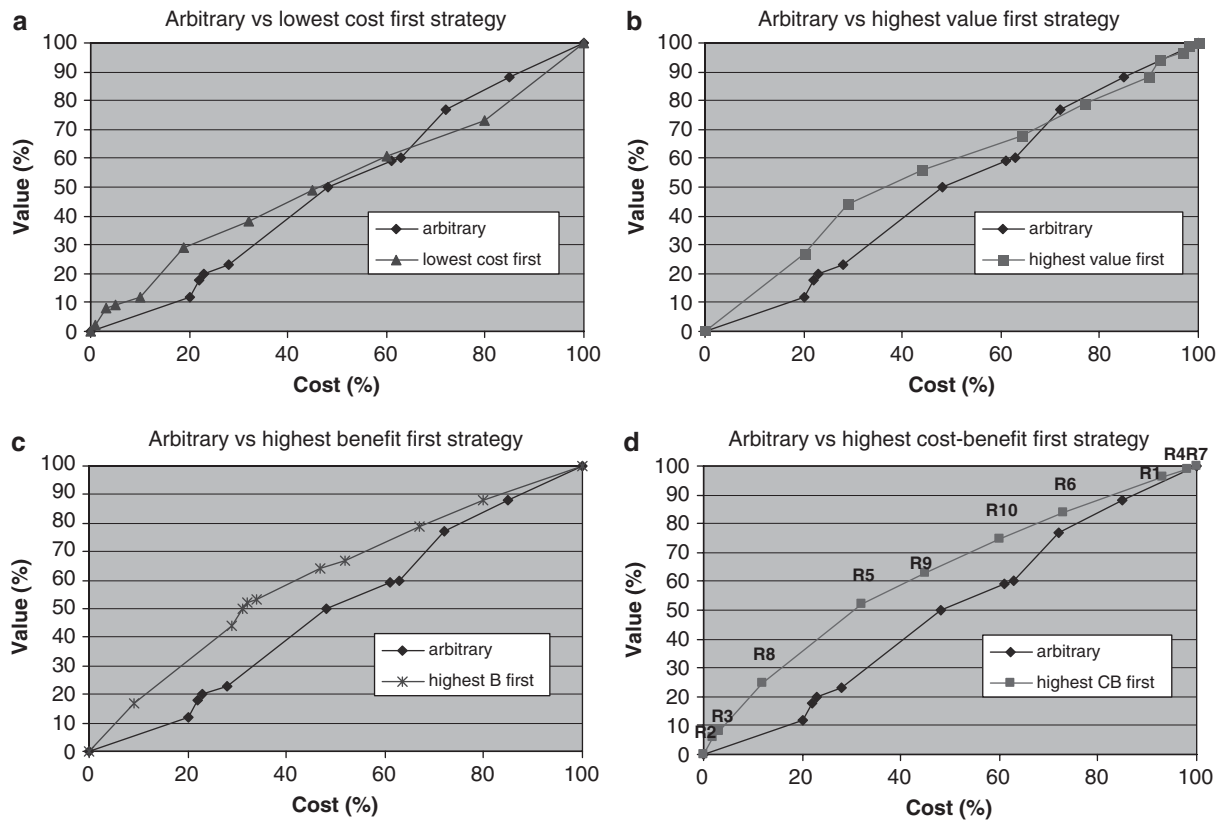


Figure 4 Cost and value strategy profile comparisons.

no dynamism (i.e., requirements volatility). One can see that dynamism will greatly affect strategy performance. The agile approach seeks to avoid extensive up-front planning and incorporate frequent reprioritization within small development iterations to allow rapid adaptation to volatile requirements. This approach too has its strategic limitations, which will be elaborated in the subsequent section on simulation. In this section, we give a detailed example of performing the HY method while analyzing its performance when there is some dynamism.

Starting with Table 2, we assume that only requirements R_1 – R_7 are the 'base' requirements. However, it would be more realistic to assume that in Step 2 of the HY method, the taskforce would likely have identified in advance some of the requirements (i.e., R_8 – R_{10}) that perhaps would not have been discovered in performing the AG or AG2 strategy. Steps 5–7 now begin with defining iteration 1 from the template provided by the taskforce. For simplicity, let us assume that no modifications are needed for the template (Step 4) and that each iteration can implement all the requirements planned in it. The following are the results of applying Steps 5–7 to our example. The gray boxes indicate the requirements that determine the iteration size.

We begin by ordering the known requirements from highest to lowest CB (Step 5) and then iteration 1 is

defined by the set of requirements with $CB > Y(S)/X(S) = 0.95$ (Step 6) implemented in the current order, that is, R_2 , R_3 , then R_5 :

Iteration 1	Requirement	R_2	R_3	R_5	R_6	R_1	R_4	R_7
	Value	6	2	27	9	12	3	1
	Cost	2	1	20	13	20	5	2
	CB	3	2	1.35	0.69	0.6	0.6	0.5

When the requirements in iteration 1 are implemented, iteration 2 begins by removing these from consideration and revisiting Step 5. Note that there is no 'carry over' of unfinished requirements as would be expected in fixed iteration strategies. During the reassessment, it is found that the value of R_4 rises to 10 and new requirements R_8 and R_9 emerge (presence of dynamism) and $Y(S)/X(S) = 0.97$ giving R_4 and R_8 for this iteration:

Iteration 2	Requirement	R_4	R_8	R_9	R_6	R_1	R_7
	Value	10	17	11	9	12	1
	Cost	5	9	13	13	20	2
	CB	2	1.9	0.85	0.69	0.6	0.5

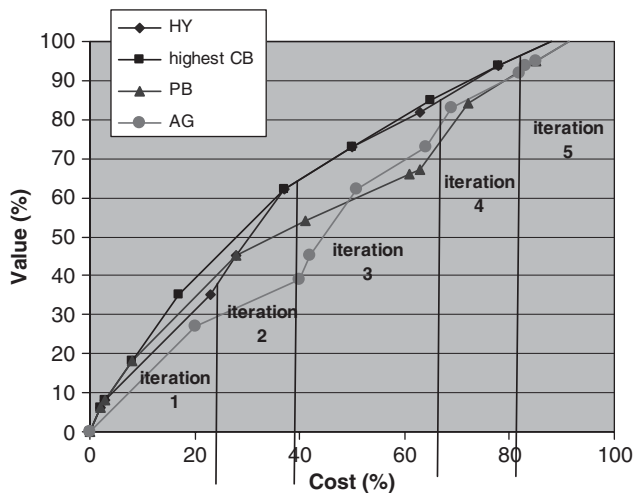


Figure 5 HY strategy profile comparison with Optimal Pareto.

R_4 and R_8 are removed and no requirements changes, so $Y(S)/X(S) = 0.69$:

Iteration 3	Requirement	R_9	R_6	R_1	R_7
	Value	11	9	12	1
	Cost	13	13	20	2
	CB	0.85	0.69	0.6	0.5

A new high-priority requirement R_{10} emerges, and now $Y(S)/X(S) = 0.68$:

Iteration 4	Requirement	R_{10}	R_1	R_7
	Value	12	12	1
	Cost	15	20	2
	CB	0.8	0.6	0.5

As $Y(S)/X(S) = 0.6$ is very close to the previous iteration value, this is the final iteration:

Iteration 5	Requirement	R_1	R_7
	Value	12	1
	Cost	20	2
	CB	0.6	0.5

Figure 5 shows the resulting strategy profile generated by the HY approach using final cost-value assessments for R_1 – R_{10} and renormalizing. To compare with PB, an example of AG is shown using the same data. We also show the Optimal Pareto strategy. Although no strategy can outperform this, we note that when dynamism is present this strategy is unachievable, as it is practically impossible to precisely predict how the final set of requirements looks like, and their final values from the

start. In spite of dynamism, the HY strategy in the example performs quite closely to the Optimal strategy. The CB strategy is fixed from the beginning and an implementation that is managed to this plan does not have the flexibility to respond to in-process requirement changes. Meanwhile, the HY method still optimizes globally after adapting to unforeseeable changes.

A simulation of mixed strategy performance

The objective of our simulation is to explore comparative properties of prioritization strategies under various conditions that are difficult or impossible to set up and observe in practice. An example of this is the 'home ground' model of Boehm & Turner (2004). In their model, PB and agile are considered as opposite extremes with respect to five project factors – *size*, *criticality*, *dynamism*, *personnel*, and *culture*. Each method has its 'home ground' at one end or the other of the scales for these factors. The theory states that most projects will not have values of these factors within either method's home ground, and thus a mixture of approaches tends to be generally more effective for any given project.

Simulation procedures for requirements evolution

For our simulation purposes, we are only interested in quantitative evaluation attributes of a requirement. The commonly used attributes for this are the *cost* of implementing (this might be effort rather than a monetary unit) and the *value* or the expected gain if the requirement is implemented. Thus, in our simulation, a requirement R_i is considered an ordered pair $(cost_i, value_i)$ where $\min_cost \leq cost_i \leq \max_cost$ and $\max_cost \leq value_i \leq \max_value$. A 'base set' of requirements $\{R_1, R_2, \dots, R_{num_reqs}\}$ is generated by assigning uniform random variables $cost_i = U(\min_cost, \max_cost)$ and $value_i = U(\min_value, \max_value)$. After each iteration, requirements volatility is handled by updating each requirement value with a normally distributed random variable $R_i = (cost_i, value_i + N(0, req_value_sigma))$, and a Poisson number of new requirements $Pois(ave_new_req_per_iter)$ are added to the base set. Cost is assumed to be non-volatile because we have found that this does not contribute an independent volatility dimension and only increases the volatility overall. If $value_i < 0$, then $value_i = 0$ from that point onwards (requirements do not raise up after being devalued). A base number of iterations num_iters and a minimum number of iterations min_iters are chosen.

The 'base' iterations determine when requirements change which is every $(total_base_cost)/number_iters$. Note that strategies do not have to use this as their iteration size. It is mainly for handling requirements volatility.

The simulation iteratively applies each requirements prioritization strategy, and when min_iters has passed, and if a Bernoulli random variable $B(end_dev_prob) = 1$, then the development is considered over, and this

completes one trial in the simulation. This simulates the unknown stopping time for a development project.

All strategies must not exceed the total cost of the base requirements at the stopping time, but may expand less due to the particular cost of the requirements left at the last iteration (i.e., if adding one more would exceed the stopping cost, then it is not used). In our investigation, we made use of default parameters abstracted by experimentation, and what appeared to be representative of 'typical' development efforts. We did not have explicit evidence to support our particular choices. However, beyond using excessive values, we have found that our results are not sensitive to any of the parameters not related to requirements volatility.

The literal view of agile and PB approaches we take simply means that we do not mix (or balance in the terminology of Boehm & Turner (2004) the activities that attributed to each respective approach. However, we most certainly desire that the activities performed within each strategy are representative of actual practice and not on specious statements, theories, or claims that have not been empirically validated. We refer to a recent empirical study of requirement practices across 16 different companies (Cao & Ramesh, 2008) to provide a real-life basis for establishing a representative simulation of agile and PB requirements prioritization strategies.

Following the conventional practice of experimentation, we made the following assumptions and simplifications in the simulation model:

- End development time is unknown
- Requirements volatility has two independent factors (λ , σ)
- Cost is non-volatile
- Change in value due to volatility is normally distributed
- Once a requirement has zero or negative value, it never regains positive value
- Arrival of new requirements due to volatility is Poisson distributed
- Single option for implementation
- Requirement implementation dependencies are not accounted for.

With the exception of naïve or arbitrary strategies (e.g., implement the requirements as they appear), all strategies rely on some form of assessments, such as cost estimation, value assessment, dependency analysis, and so forth. Costs are difficult to trace down to a particular requirement (or even within a given iteration). Value is generally 'intangible', and also not easily traced to particular requirement. Usually, overall value or value for completed groups of requirements that represent a complete set of functionality is considered. The so-called 'earned value' is not the actual value (Boehm, 2003), and may not be useful for evaluating prioritization effectiveness. Dependencies change as requirements change. As Sjöberg *et al.* (2002) points out, requirements prioritization is difficult to monitor and measure 'in vitro' (i.e., in

actual practice). Under these conditions, simulations might help verify the logic in the conceptual model (i.e., face validity), and test extreme and unlikely conditions (Sargent, 1998).

Measures of strategy effectiveness

The effectiveness of a strategy involves a number of perspectives, none of which dominates. For the purposes of the simulation, we have developed a number of 'goodness-of-strategy' metrics (Table 3).

Simulation results

With the exception of the first two assumptions enumerated earlier (i.e., *end development time is unknown and requirements volatility has two independent factors*), we have found that our results were unaffected when these were changed or relaxed. For example, we experimented with non-normal and non-Poisson distributions with no change in results. This in and of itself is somewhat surprising, especially the last assumption – ignoring implementation dependencies. We have studied this last assumption in some detail both in theory and by conducting experiments that accounted for varying degrees of dependencies (using so-called dependency graphs) – both extreme and mild – and have observed no substantial difference in simulation behavior or results. One possible explanation would be that, for some prioritization properties, requirements dependencies do matter, and we are not claiming otherwise here. For our purposes, however, requirements dependencies will, as with the other assumptions, affect all strategies equally and as such do not affect the properties we are investigating, i.e., requirements dependencies are independent of prioritization strategy. A detailed description of the simulation can be found in Port *et al.* (2008), and the simulation source code is available upon request from the authors.

In this simulation, the most relevant factor for requirements prioritization is dynamism, defined as the percentage of requirements change per month. According to Boehm & Turner (2004), agile methods have a home ground at 50% or more, whereas PB has at 1% or less. This translates straightforwardly into our simulation requirements volatility parameters as indicated in Table 4.

Our simulation results are consistent with the above with very high confidence. Specifically, there is a significant difference in ranks in all measures except tc in the (high λ , high σ) case where PB is slightly better. That PB has lower cost in this case is not unexpected given the last requirement implemented in this strategy is likely to have much lower cost than the requirement that would have been implemented had development not ended. This PB has greater 'pullback' at the end than AG.

Although the home ground theory provides no expectations for the (high λ , low σ) and (low λ , high σ) cases, agile generally is better on all measures, but not significantly better. Significance is defined as having

Table 3 Goodness-of-strategy metrics

Metric	Definition	Explanation
<i>tv</i>	Total value of the requirements implemented	Represents the value on the y-axis at the end point of a strategy (or the ultimate 'height'). Only the values of the requirements at the end are used, not during the implementation, as these change over the course of the project and the end values represent, presumably, what is actually delivered
<i>tc</i>	Total cost of the requirements implemented	Requirement costs are assumed to be constant throughout the project. This represents the value on the x-axis at the end point of a strategy (or the ultimate 'length')
<i>CB</i>	Cost–benefit ratio	Is defined as tv/tc
<i>Ben</i>	Benefit	Is defined as $tv - tc$
<i>Int</i>	Integral	Is the discreet integral or total area under the strategy curve. This represents the 'total value created' by a strategy with respect to the cost. This is different than total value of the requirements implemented. It is related to the economic risk of the strategy
<i>Fr</i>	Frontier ratio	This is defined as the ratio $integral/(integral \text{ for frontier strategy up to same cost})$ where the <i>frontier strategy</i> is the curve generated the optimal values of the requirements at the development end cost. The frontier strategy is 'Optimal', but is generally an unachievable strategy due to the random changes in values and new requirements added during the iterations (i.e., one cannot precisely predict the future). The closer <i>fr</i> is to 1, the more closely it resembles the ideal frontier strategy up to its <i>TotCost</i> . What makes this measure attractive is that it accounts for the 'goodness' of a strategy overall rather than just at the end. That is, a strategy may perform poorly early on, but jump up at the end to deliver a good end value. However, if the development had stopped earlier, it would have actually performed poorly. Another way to look at this is that as no strategy, in theory, can outperform (or exceed) the frontier, the closer a strategy resembles frontier strategy the 'better' that strategy is. The frontier strategy has the obvious property that it has the maximum possible area under it, so the larger a strategy's integral is, the closer it must be to the frontier strategy

Table 4 Dynamism parameters' home grounds

Average new requirements per iteration (λ)	HIGH	-	Agile (AG)
	LOW	Plan-Based (PB)	-
		LOW	HIGH
		Requirements value standard deviation (σ)	

more than 25% difference in the average rank for a measure after 'convergence' of the simulation (discussed later).

Visualization of simulated prioritization strategies

We begin by visualizing a single simulation run under a typical dynamism scenario, and then the two extremes to both validate the expected behavior of the simulation for AG and PB, and observe the adjustability properties of the new AG2 and HY strategies. Figure 6 is a single run with $\lambda = 1.4$ and $\sigma = 15\%$ (medium dynamism).

Each of the curves in the figure represents the results of applying a given strategy to the final requirements (with the exception of the Optimal initial curve). Each point represents the implementation of a requirement, at which time, its cost is added to the cumulative total and its value is added to the cumulative value up to this point. Hence, each curve is always increasing.

To explain the features in Figure 6, the solid curve is the 'Optimal initial' strategy, and it is the Pareto plot of the base requirements. If there is no dynamism, this would be the Optimal strategy as no other order could have higher value at lower cost than this curve at any point. The dashed curve depicts the 'Optimal Frontier' that represents the Pareto plot of the complete final set of requirements at the time development ends. This includes the base set plus any requirements added during the development and their respective value changes. No strategy can theoretically rise above this curve as it represents the Optimal strategy given complete knowledge of all the requirements and their values in advance. Such a strategy is impossible to achieve because of the stochastic nature of dynamism. The more similar a strategy is to the Optimal Frontier, the better is its performance, and hence the motivation for the *fr* measure. The difference between the 'Optimal initial'

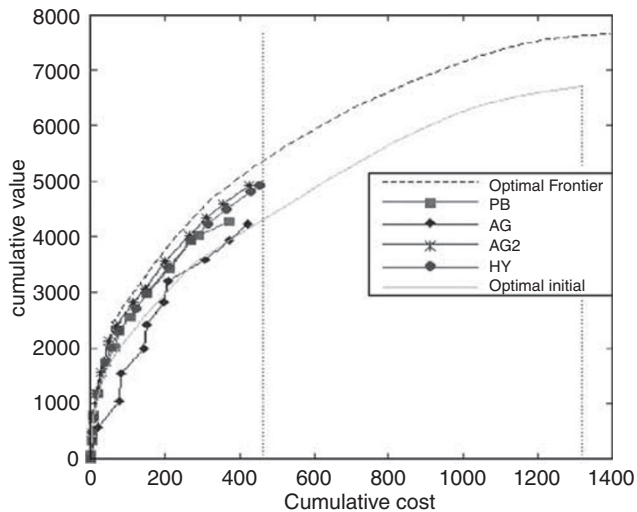


Figure 6 Medium dynamism simulation run.

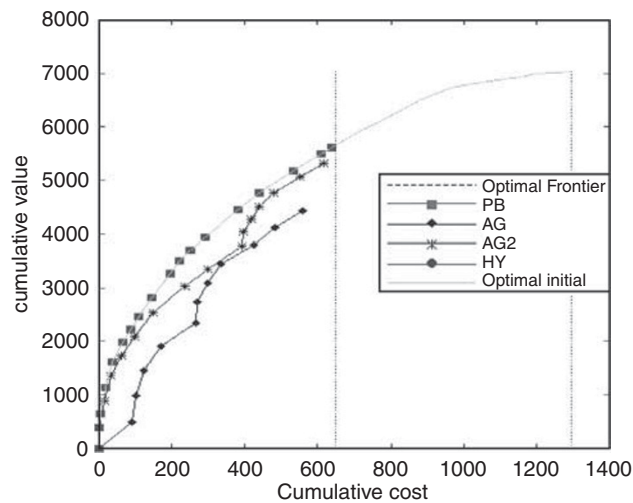


Figure 7 No to low dynamism simulation run.

and 'Optimal Frontier' refers to the degree of dynamism for a given run, that is, the farther apart these are, the greater the dynamism. The tall vertical line indicates the maximum total cost of the development at the stopping time. No strategy may exceed this cost, and so no curve may extend beyond this line.

Note that all the strategies are consistent with the features described. This verification has been performed hundreds of times under a large variation in simulation parameters with no unexpected behavior. We observe that neither AG nor PB perform particularly well as is predicted by the home ground theory for medium dynamism. The AG2 and HY perform best and at about the same level.

Figure 7 is a single run with $\lambda = 1/1000$ and $\sigma = 1/10\%$ (low dynamism). For this scenario, we would expect that

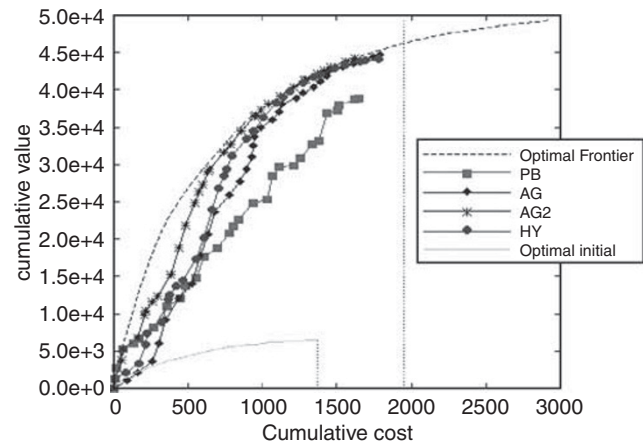


Figure 8 High dynamism simulation run.

as there is nearly zero dynamism, the 'Optimal initial' and 'Optimal Frontier' would be identical. This is clearly the case in Figure 4, as these two curves completely overlap. With no dynamism, the PB strategy is predicted to be the best and AG the worst. In fact, PB should be identical to the Optimal initial up to the stopping cost. This is indicated in the figure above, but is difficult to see as it is covered over by the HY strategy. This latter observation is notable as it indicates that the HY strategy has precisely adapted to the expected best strategy PB. This result has been seen to hold in general and so we have confidence that the HY is adaptable to dynamism. It is also notable that the AG2 strategy also performs reasonably well here and also appears to be adaptable. A more subtle observation is that the AG2 and AG strategies both clearly show the 'diminishing returns' characteristic within each iteration whereby the values sharply increase at the beginning of each new iteration. This is consistent with the expected behavior of the agile approach that uses fixed iteration sizes.

Figure 8 is a single run with $\lambda = 20$ and $\sigma = 200\%$ (very high dynamism).

With many new requirements and large value swings, we expect the large difference between the Optimal initial and Optimal Frontier as seen in Figure 8. As expected, AG would be the best and PB the worst strategy. In this simulation run, all the strategies that reprioritized within each iteration seemed to perform equally well (at least at the end development time). This is not generally the case, and even here closer inspection will reveal significant differences in these strategies. For example, if the development ended much earlier, say at 500, then the AG2 strategy would be the clear winner. Indeed, it is consistently closer to the Optimal Frontier than the other strategies. What is notable is the similarity of the AG2 and HY strategies. Both strategies appeared to adapt well at this extreme level of dynamism, in this case better than AG. In general, this case is unlikely.

Discussion: comparison of strategy performances

To appreciate the properties of the strategies in general, we considered the average ranks and standard deviations over 1000 simulation runs for the six strategy measures described earlier. We chose 1000 runs because, uniformly, the average values converged to at least 1 decimal of precision for all the measures at this level. The rank for a measure M on a given run is the (number of strategies being compared)–(number of strategies with ‘worse’ measures within the specified tolerance). For example, if there are four strategies being compared and the agile strategy has a tv that is more than 5% (the tolerance level) greater than the other three, then it will have rank 1. It is unjustifiable to consider two values that are within a given small percentage of each other to have different ranks, hence the use of a tolerance value (we use 5%) to create rank equivalences. Thus, it is possible for all strategies to have the same rank indicating no substantial differences for that measure. Table 5 is an example of average rank results for 1000 runs with $\lambda = 1.4$ and $\sigma = 15\%$ (medium dynamism).

The results are significant to one decimal place. It can be observed that the HY strategy is the top rank for all but tc (as is expected from our earlier discussion on cost and PB). Also, notable is that HY has the lowest standard deviation for all measures it is top ranked. This can be interpreted as the degree to which we expect a typical simulation run to be near its average rank value. That is, the HY strategy is more consistently top ranked than the other strategies. It is not top ranked on average because it swings wildly from top to bottom rank.

Table 6 considers the top average ranks from 1000 random trials under different levels of dynamism. The PB strategy wins on tc mostly due to the abrupt stopping rule, which is that no requirements can be implemented that exceed the cost at the stopping point. What happens with the PB strategy is that at the stopping point the planned set of requirements generally cannot exactly meet the stopping cost (no rearrangements are possible in the PB strategy, even at the end), so there is no choice in which requirement to end on – the one which makes the cumulative cost less than the stopping point whose next requirement in the planned order would make a cumulative cost exceed the stopping point. As the requirements are ordered by CB, the farther the stopping point is, the greater the cost per requirement is likely to be. So, the end requirement is more likely to ‘pull back’ far behind the stopping point cost because the next requirement is likely to have a higher cost. Because of this, total cost is probably not a very useful ‘goodness’ of strategy measure.

In the overall spectrum of performance, HY dominates in four areas, is very strong in another two, and is strong but not dominant in one additional area – a total of seven of the nine scenarios considered. It is worth noting that the strongest areas are when the dynamism is medium in either λ or σ and curiously when λ is low and σ is high. The dominance of HY is summarized in Table 6 by

Table 5 Average ranks

	<i>tv</i>	<i>tc</i>	<i>Int</i>	<i>Ben</i>	<i>CB</i>	<i>Fr</i>
PB	1.67	1.18	1.82	1.74	1.49	1.90
SD	1.04	0.46	0.97	1.08	0.87	0.86
AG	2.34	1.26	3.43	2.45	2.41	3.56
SD	1.32	0.57	0.94	1.32	1.32	0.84
AG2	1.18	1.21	1.42	1.20	1.16	1.34
SD	0.55	0.51	0.70	0.57	0.48	0.62
HY	1.07	1.23	1.23	1.09	1.10	1.21
SD	0.30	0.53	0.51	0.35	0.40	0.47

^a $n = 1000$, $\lambda = 1.4$, $\sigma = 15\%$.

Table 6 Best average ranks

High $\lambda = 20$	Value: AG2 Cost: PB Integral: AG2 Ben: AG2 CB: AG2 Fr: AG2	Value: AG2 Cost: PB Integral: AG2 Ben: AG2 CB: AG2 Fr: AG2	Value: AG Cost: PB Integral: HY Ben: AG CB: HY, AG Fr: HY
Medium $\lambda = 1.4$	Value: HY Cost: PB Integral: HY Ben: HY CB: HY Fr: HY	Value: HY Cost: PB Integral: HY Ben: HY CB: HY Fr: HY	Value: HY Cost: PB Integral: HY Ben: AG CB: HY Fr: HY
Low $\lambda = 0$	Value: HY, PB Cost: PB Integral: HY Ben: HY, PB CB: PB Fr: HY, PB Low $\sigma = 0\%$	Value: HY Cost: PB, AG2 Integral: HY Ben: HY CB: HY Fr: HY Medium $\sigma = 15\%$	Value: HY Cost: HY Integral: HY Ben: HY CB: HY Fr: HY High $\sigma = 200\%$

^aHY: hybrid, PB: plan-based, AG: agile, AG2: agile cost-benefit.

^b $n = 1000$, $\lambda = 1.4$, $\sigma = 15\%$.

shading where darker shades indicate greater HY dominance.

Conclusion

The home ground theory of Boehm & Turner (2004) suggests that for requirements prioritization on projects with medium dynamism (or volatility), a mix of agile and PB methods would be the most suitable strategy. Our simulation results indicate that the mixed methods of AG2 and HY outperform non-mixed methods PB and AG are thus consistent with this theory. Given that volatility (dynamism) is unlikely to be at either a low or high extreme and generally is unknown or non-constant, the practical implication of this result is that it is risky to commit to a strictly PB or agile strategy. For this reason, we infer that in practice, mixed strategies are most widely used. However, it is not clear that simply any mixed strategy will be effective. We propose a seven-step model

to look for an appropriate balanced and effective strategy. We have derived two mixed strategies – AG2 that takes a majority agile approach and adds a bit of PB by including cost assessment and Pareto ordering in the prioritization, and HY that takes primarily a PB approach but adds frequent (but heavily guided) reprioritization. As suggested by our simulation, both AG2 and HY adapt well to whatever the dynamism level is (i.e., low, medium, and high). The HY strategy appears to be the overall best performer, but the price for this approach is greatly increased prioritization effort as it includes the major activities of both the PB and AG strategies. Although AG2 did not perform as well as HY overall, it significantly outperformed both AG and PB. It requires less effort than PB and only a little more effort than AG to incorporate cost assessment and Pareto planning within each iteration.

Thanks to the simulation, we gain additional confidence that mixed strategies are likely to yield better results than ‘pure’ agile or PB approaches, based on a number of different metrics. As we have given quite a bit of attention on the impacts of volatility (i.e., no, low, medium, and high dynamisms), we have been able to outline tradeoffs in using different prioritization strategies.

There are some possible threats to the validity of the simulation results. First, the representations of agile and PB methods are generic. Our simulation was derived from real-life case studies of RE and agile practices (e.g., Cao & Ramesh, 2008). There are, of course, many variations in actual individual practices, and we have not exhaustively represented all such variations. We have abstracted the fundamental characteristics in their respective approaches to prioritization from the empirical studies, and believe that these are generally representative enough so that no particular practice would produce significantly different results. In the literature, agile and PB approaches are described ‘prescriptively’ and are likely overly idealistic to be implemented strictly as specified. It is suspected that for many of the reasons discussed in this paper, practitioners generally used mixed strategies. Our simulation provides some simulation-based reasoning to recommend this practice and guidance in choosing and implementing a mixed strategy using the proposed step-by-step process model.

Another possible area of concern is the simplicity in defining volatility or dynamism in the simulation. However, the rather unambiguous results give us a sense of confidence in the argumentation. Likewise, one might argue that the simulation assumptions could have been misrepresenting the reality. We have experimented with a large variety of different assumptions (including adding requirements dependencies) with no change in results. The assumptions considered were abstracted from a variety of empirical studies on requirements prioritization. Thus, we believe that the assumptions used currently are reasonable. We also plan to conduct further simulation exploring the impact of mixed

strategies to incorporate lower and upper limits on iteration length.

Without a detailed study of a large number of projects, it is difficult to ascertain what parameter settings could reliably be considered realistic. As such, the simulation parameters used in our study could have been unrealistic. To address this problem, we have used a wide variation in parameter settings and, again, observed no change in results. Thus, we believe that if we had had more realistic parameter values, our results would have not changed.

Currently, we are extending our investigation to strategies that include implementation options. That is, each requirement may have multiple ways to be implemented with differing value and cost combinations. Options may consist of making use of different techniques or technologies, or economic options, such as partial requirement implementation. Such options open up a new dimension for strategic prioritization enabling expanded trade-off considerations that may increase strategy effectiveness. Consideration of options will not obviate our current results as options do not directly affect dynamism. Indeed, options are a natural generalization of the single implementation assumption that we currently use. We have been limited in pursuing this area due to a lack of empirical data on options consideration in practice. Because of this, we plan to pursue such an empirical study to use as a basis for extending our simulation. As most empirical studies of practice focus narrowly on either agile or PB approaches, we wish to concentrate on the use and effectiveness of mixed strategies. To link our work to real-life projects, we plan to create an online system in which both agile and PB practitioners can log in to enter data from their real-life projects. In return, the system would provide them with feedback and suggestion to implement the mixed strategy, and provide us with empirical means to validate our approach.

In summary, we conducted a comprehensive simulation under a variety of conditions of requirements dynamism and variable project size and duration. Our results suggest that a mixed strategy for requirements prioritization seems to work best in all but cost for typical levels of dynamism on average. Findings also indicate that, as theorized, PB and agile strategies perform well within opposite extremes of dynamism. It is hoped that this paper will stimulate additional investigations of mixed agile and PB approaches applied to other software development management areas.

Acknowledgements

The authors thank Alistair Cockburn and Philip Johnson for their suggestions and comments on an earlier version of this paper. We are indebted to the anonymous reviewers and the editors. Their constructive comments have significantly helped improve the clarity and quality of the arguments made in this paper.

About the authors

Dan Port is Professor of Information Technology Management at the Department of Information Technology, Shidler College of Business, University of Hawaii at Manoa. His research interests are in software assurance, strategic software engineering, and strategic IT management.

Tung Bui is Matson Navigation Company Professor at the Department of Information Technology, Shidler College of Business, University of Hawaii at Manoa. His current research interests include effective use of information technology in organizations and the design and implementation of negotiation support algorithms for e-commerce and supply chains applications.

References

- ABRAHAMSSON P, SALO O, RONKAINEN J and WARSTA J (2002) *Agile Software Development Methods: Review and Analysis* Vol. 478 VTT Publications p. 478.
- AURUM A and WOHLIN C (2003) The fundamental nature of requirements engineering activities as a decision-making process. *Information and Software Technology* **45**, 945–954.
- BALCI O (1995) Principles and Techniques of simulation validation, verification and testing. In *Proceedings of the 27th Conference on Winter Simulation*, pp 147–154, IEEE Computer Society, Washington DC.
- BECK K (2001) *Extreme Programming: Explained* 7th edn, Addison-Wesley, Boston.
- BOEHM B (1991) Software risk management: Principles and practices. *IEEE Software* **8**(1), 32–41.
- BOEHM B (2003) Value-based software engineering. *ACM SIGSOFT Software Engineering Notes* **28**(2), 4.
- BOEHM B and TURNER R (2004) Balancing agility and discipline: evaluating and integrating agile and plan-driven methods. In *Proceedings of the 26th International Conference on Software Engineering*, pp 718–719, IEEE Computer Society, Washington DC.
- BUI T (1987) Co-op: a Multiple-Criteria Group Decision Support System for Cooperative Decision Making. In *Lecture Notes in Computer Science*. Springer Verlag.
- CAO L and RAMESH B (2008) Requirements engineering practices: an empirical study. *IEEE Software* **25**(1), 60–67.
- CESCHI M, SILLIT A, GIANCARIO S and DE PANFILIS S (2005) Project management in plan-based and agile companies. *IEEE Software* **22**(3), 21–27.
- COCKBURN A (2000) *Writing Effective Use Cases*. Addison-Wesley, Boston.
- COHEN M (2005) *Agile Estimating and Planning*. Prentice Hall PTR, Englewood Cliffs, NJ.
- DAMIAN DE and ZOWGHI D (2002) The impact of stakeholders' geographical distribution on managing requirements in a multi-site organization. In *Proceedings of the 10th Anniversary IEEE Joint International Conference on Requirements Engineering*, pp 319–330, IEEE Computer Society, Washington DC.
- HAMMOND KJ and BURKE RD (1997) A plan-based approach to information agents. Department of Defense report, Department of Computer Science, University of Chicago, Chicago, IL.
- HOFMANN HF and LEHNER F (2001) Requirements engineering as a success factor in software projects. *IEEE Software* **18**(4), 58–66.
- IN HP, OLSON D and RODGERS T (2002) Multi-criteria preference analysis for systematic requirements negotiation. In *Proceedings of the twenty-sixth International Computer Software and Applications Conference on Prolonging Software Life: Development and Redevelopment*, pp 887–892, IEEE Computer Society, Washington DC.
- KARLSSON J (1996) Software requirements prioritizing. In *Proceedings of the second International Conference on Requirements Engineering*, p 110, IEEE Computer Society, Washington DC.
- KARLSSON J and RYAN K (1997) A cost-value approach for prioritizing requirements. *IEEE Software* **14**(5), 67–74.
- KLEINDORFER GB, LEINDORFER GB and GANESHAN R (1993) The philosophy of science and validation in simulation. In *Proceedings of the twenty-fifth conference on Winter Simulation*, pp 50–57, ACM, New York, NY.
- LEHTOLA L, KAUPPINEN M and KUJALA S (2004) Requirements prioritization challenges in practice. In *Proceedings of 5th International Conference on Product Focused Software Process Improvement*, pp 497–508, Kansai Science City, Japan.
- LUBARS M, POTTS C and RICHTER C (1993) A review of the state of the practice in requirements modeling. In *Proceedings of the IEEE International Symposium on Requirements Engineering*, pp 2–14, IEEE Computer Society, Washington DC.
- MOISIADIS F (2002) The fundamentals of prioritising requirements. In *Proceedings of the Systems Engineering, Test and Evaluation Conference*, pp 108–119, Brisbane, Australia.
- OREN TI (1981) Concepts and criteria to assess acceptability of simulation studies: a frame of reference. *Communications of the ACM* **24**(4), 180–189.
- PAETSCH F, EBERLEIN A and MAURER F (2003) Requirements engineering and agile software development. In *Proceedings of the Twelfth International Workshop on Enabling Technologies: Infrastructure For Collaborative Enterprises*, p 308, IEEE Computer Society, Washington DC.
- PATTON J (2008) How I stopped worrying and learned to love prioritization. [WWW document] <http://www.stickyminds.com/sitewide.asp?Function=WEEKLYCOLUMN&ObjectId=14004&ObjectType=ARTCOL&btntopic=artcol>.
- POPPENDIECK M and POPPENDIECK T (2003) *Lean Software Development: An Agile Toolkit*. Addison-Wesley, Longman Publishing Co. Inc., Boston, MA.
- PORT D, KAZMAN R and NAKAO K (2007) Practicing what is preached: 80–20 rules for strategic IV&V assessment. In *Proceedings of IEEE Conference on Exploring Quantifiable Information Technology Yields (EQUITY)*, Amsterdam, Netherlands.
- PORT D, OLKOV A and MENZIES T (2008) Using simulation to investigate requirements prioritization strategies. In *Proceedings of the conference on Automated Software Engineering*.
- RAMESH B, CAO L, MOHAN K and XU P (2006) Can distributed software development be agile? *Communications of the ACM* **49**(10), 41–46.
- REGNELL B, HOST M, NATTOCH DJ, BEREMARK P and HJELM T (2001) An industrial case study on distributed prioritization in market-driven requirements engineering for packaged software. *Requirements Engineering* **6**(1), 51–62.
- SAATY TL (1990) *Multicriteria Decision Making: The Analytic Hierarchy Process*. RWS Publications, Pittsburgh.
- SARGENT R (1998) Verification and validation of simulation model. In *Proceedings of the Thirteenth Conference on Winter Simulation*, pp 121–130, IEEE Computer Society, Washington DC.
- SIVZATTIAN A and NUSEIBEH B (2001) Linking the selection of requirements to market value: A portfolio-based approach. In *The 7th International Workshop on Requirements Engineering*, ACM, Interlaken, Switzerland.
- SJOBERG D, ANDA B, ARISHOLM E and DYBA T (2002) Conducting realistic experiments on software engineering. In *Proceedings of the International Symposium on Empirical Software Engineering*, IEEE Computer Society, Washington DC.
- WIEGERS K (1999) First things first: prioritizing requirements. *Software Development* **7**(9), 11–19.